

Programmieren für Kids

JavaScript für Kids

Modul 6

Timer: Dinge passieren von selbst



Trainer: Wolfgang Schedl

Kurz zurückgeblickt

Bis jetzt passiert alles nur, **wenn du klickst**. Die Seite wartet still, bis du etwas tust. Heute lernst du **Timer**. Damit passiert etwas **von selbst**: es tickt, es zählt runter, es taucht auf. Genau das macht aus einer Übung ein echtes Spiel.

Das baust du in diesem Modul

- ein **Reaktionsspiel**: wie schnell bist du?
- eine **Klick-Jagd**: so viele Klicks wie möglich, bevor die Zeit abläuft
- als Zugabe: dein Klick-Imperium verdient jetzt **von selbst**

Zum ersten Mal läuft die Zeit gegen dich.



Kapitel 1

Timer: von selbst

Etwas später: setTimeout

setTimeout macht etwas **einmal**, nach einer Wartezeit:

```
setTimeout(sagHallo, 2000)    // nach 2000 ms

function sagHallo() {
  alert("Hallo!")
}
```

Die Zeit ist in Millisekunden: 1000 ms = 1 Sekunde, 2000 ms = 2 Sekunden.

Millisekunden

Timer rechnen in Millisekunden (ms). 1000 davon sind eine Sekunde:

100	=	blitzschnell
500	=	eine halbe Sekunde
1000	=	eine Sekunde
5000	=	fünf Sekunden



Wichtig: Name ohne Klammern

Du gibst dem Timer den **Namen** der Funktion, ohne Klammern:

```
setTimeout(sagHallo, 2000)    // richtig  
setTimeout(sagHallo(), 2000)  // FALSCH
```

Mit Klammern rufst du die Funktion *sofort* auf. Ohne Klammern gibst du sie dem Timer, der sie *später* aufruft.



Immer wieder: setInterval

`setInterval` macht etwas **immer wieder**, in festen Abständen:

```
function tick() {  
  console.log("tick")  
}  
  
setInterval(tick, 1000)  // jede Sekunde
```

Alle 1000 ms läuft `tick` erneut, von selbst, ohne dass du klickst.

■ Stoppen: clearInterval

Ein `setInterval` läuft ewig, bis du ihn stoppst. Dafür merkst du ihn dir in einer Variable:

```
let timer = setInterval(tick, 1000)

clearInterval(timer)    // stoppt den Timer
```

Ohne das Merken in `timer` könntest du ihn nie wieder anhalten.

Übung: der Zähler läuft von selbst



ÜBUNG

ca. 12 Minuten

Baue einen Zähler, der jede Sekunde von selbst hochzählt:

```
let zahl = 0
function hoch() {
  zahl = zahl + 1
  document.getElementById("anzeige").innerHTML = zahl
}
setInterval(hoch, 1000)
```

★ **Bonus:** Stoppe bei 10 mit `clearInterval`.

Fehler-Detektive



FINDE DEN FEHLER

Dieser Timer läuft nicht jede Sekunde, sondern nur ein einziges Mal am Anfang. Warum?

```
setInterval(tick(), 1000)
```

Tipp: Was machen die Klammern hinter tick?



Projekt

Das Reaktionsspiel



Jetzt öffnen: [6a-reaktionsspiel](#)

in VS Code – oder auf der Kursseite

⚡ Wie es geht

- Du drückst „Start“, und ein Feld wird rot
- nach einer **zufälligen** Zeit wird es grün
- sobald es grün ist, klickst du so schnell du kannst
- das Spiel misst deine **Reaktionszeit**

Neu ist nur: die Zeit messen. Der Rest ist Timer plus das, was du schon kannst.



So ist deine Vorlage aufgebaut

```
<div id="feld" onclick="klick()">Drück auf Start</div>  
<button onclick="starte()">Start</button>
```

Ein großes Feld **feld** (das man anklicken kann) und ein Start-Knopf.

Warten mit setTimeout

„Start“ färbt das Feld rot und wartet eine zufällige Zeit, dann wird es grün:

```
function starte() {  
  document.getElementById("feld").innerHTML = "Warte auf  
  Grün..."  
  const wartezeit = Math.floor(Math.random() * 3000) + 1000  
  setTimeout(gruen, wartezeit)  
}
```

Die Wartezeit ist 1000 bis 4000 ms, also jedes Mal anders.



Grün werden und Zeit merken

```
let startZeit = 0

function gruen() {
  document.getElementById("feld").innerHTML = "JETZT klicken!"
  document.getElementById("feld").style.backgroundColor =
    "green"
  startZeit = Date.now()
}
```

`Date.now()` gibt die aktuelle Uhrzeit in Millisekunden. Wir merken sie uns als Startpunkt.

Reaktionszeit messen

Beim Klick schauen wir, wie viel Zeit seit dem Grün vergangen ist:

```
function klick() {  
  const zeit = Date.now() - startZeit  
  document.getElementById("feld").innerHTML = "Deine Zeit: "  
    + zeit + " ms"  
}
```

„Jetzt“ minus „Startpunkt“ = deine Reaktionszeit. Unter 250 ms ist richtig schnell!

Übung: dein Reaktionsspiel



PROJEKT

ca. 15 Minuten

- Bring das Reaktionsspiel zum Laufen
- Spielt gegeneinander: wer ist am schnellsten?

★ **Bonus:** Sag „Zu früh!“, wenn jemand klickt, bevor das Feld grün ist.

⚡ Zeig dein Reaktionsspiel!

JETZT klicken!

Deine Zeit: 212 ms



Projekt

Die Klick-Jagd



Jetzt öffnen: 6b-klick-jagd

in VS Code – oder auf der Kursseite

Wie es geht

- du hast **15 Sekunden** Zeit
- klick so oft du kannst auf den Knopf
- ein Countdown zählt die Sekunden herunter
- bei 0 ist Schluss, und du siehst deine Punkte

Das Herzstück ist ein `setInterval`, der jede Sekunde herunterzählt.



So ist deine Vorlage aufgebaut

```
<h2 id="zeit">Zeit: 15</h2>
<h2 id="punkte">Punkte: 0</h2>
<button onclick="start()">Start</button>
<button onclick="klick()">Klick!</button>
```

Anzeigen für **zeit** und **punkte**, ein Start-Knopf und der Klick-Knopf.

Der Countdown startet

```
let punkte = 0
let zeit = 15
let timer

function start() {
  timer = setInterval(tick, 1000)
}
```

„Start“ lässt den Timer laufen: ab jetzt läuft tick jede Sekunde von selbst.

Jede Sekunde weniger

```
function tick() {  
  zeit = zeit - 1  
  document.getElementById("zeit").innerHTML = "Zeit: " + zeit  
  if (zeit <= 0) {  
    clearInterval(timer)  
    document.getElementById("zeit").innerHTML = "Vorbei! Punkte: "  
    + punkte  
  }  
}
```

Bei `zeit <= 0` stoppen wir den Timer mit `clearInterval` und zeigen das Ergebnis.

Klicken zählt

```
function klick() {  
  punkte = punkte + 1  
  document.getElementById("punkte").innerHTML = "Punkte: " +  
    punkte  
}
```

Solange die Zeit läuft, bringt jeder Klick einen Punkt. Das kennst du schon, neu ist nur der Countdown daneben.

Übung: deine Klick-Jagd



PROJEKT

ca. 15 Minuten

- Bring die Klick-Jagd zum Laufen
- Tragt eure Rekorde an der Tafel ein

★ **Bonus:** Merke dir den besten Wert in einer Variable und zeig ihn als „Rekord“ an.

Zeig deine Klick-Jagd!

Zeit: 7

Punkte: 41

Klick schnell!



Bonus

**Automatik fürs
Klick-Imperium**

∞ Punkte von selbst

Erinnerst du dich ans Klick-Imperium? Jetzt verdient es auch **ohne** Klicken:

```
let proSekunde = 1

function autoKlick() {
  punkte = punkte + proSekunde
  zeigeAlles()
}

setInterval(autoKlick, 1000)
```

Jede Sekunde kommen proSekunde Punkte dazu, ganz von allein.

∞ Ein Auto-Upgrade

Jetzt kannst du ein neues Upgrade bauen, das nicht proKlick, sondern proSekunde erhöht.

- kaufe einen „Helfer“, der proSekunde um 1 erhöht
- schon arbeitet dein Imperium für dich, während du zuschaust

Genau so funktionieren die großen Klick-Spiele im Internet.

✓ Das kannst du jetzt

- mit `setTimeout` etwas **einmal später** tun
- mit `setInterval` etwas **immer wieder** tun
- einen Timer mit `clearInterval` **stoppen**
- mit `Date.now()` die **Zeit messen**

Jetzt läuft deine Seite von selbst. Das ist der Sprung zum echten Spiel!

☰ Neue Befehle auf einen Blick

setTimeout(fn, ms)	=	einmal später tun
setInterval(fn, ms)	=	immer wieder tun
clearInterval(timer)	=	den Timer stoppen
Date.now()	=	die Zeit in ms

Den Funktionsnamen immer ohne Klammern übergeben.



Als Nächstes: Modul 7

Deine Spiele laufen jetzt von selbst. Als Nächstes steuerst du sie:

Bewege eine Figur mit den Pfeiltasten.

Dann fühlt es sich endgültig wie ein echtes Computerspiel an.